

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина «Стандартизация программного обеспечения»

ОТЧЕТ
по лабораторной работе № 1

на тему:

МЕТРИКИ РАЗМЕРА ПРОГРАММ

Выполнили
Студенты гр. 451001

Држевецкий Н. А.
Клевко А. А.

Руководитель, преподаватель

Болтак С. В.

Минск, 2026

Код анализируемой программы

```
#!/usr/bin/perl
use strict;
use warnings;

sub tokenize {
    my ($expr) = @_;
    my @tokens;
    while ($expr =~ /\S/g) {
        if ($expr =~ /\G(\d+(?:\.\d+)?)/gc) {
            push @tokens, $1;
        } elsif ($expr =~ /\G([\+\-\*\^\(\)])/gc) {
            push @tokens, $1;
        } else {
            my $pos = pos($expr) || 0;
            die "Ошибка: неверный символ '" . substr($expr, $pos, 1) . "' на позиции $pos\n";
        }
    }
    return \@tokens;
}

sub parse_expr {
    my ($tokens) = @_;
    my $val = parse_term($tokens);
    while (@$tokens && ($tokens->[0] eq '+' || $tokens->[0] eq '-')) {
        my $op = shift @$tokens;
        my $rhs = parse_term($tokens);
        $val = ($op eq '+') ? $val + $rhs : $val - $rhs;
    }
    return $val;
}

sub parse_term {
    my ($tokens) = @_;
    my $val = parse_factor($tokens);
    while (@$tokens && ($tokens->[0] eq '*' || $tokens->[0] eq '/')) {
        my $op = shift @$tokens;
        my $rhs = parse_factor($tokens);
        die "Ошибка: деление на ноль\n" if $op eq '/' && $rhs == 0;
        $val = ($op eq '*') ? $val * $rhs : $val / $rhs;
    }
    return $val;
}

sub parse_factor {
    my ($tokens) = @_;
    die "Ошибка: незавершённое выражение\n" unless @$tokens;
    my $token = shift @$tokens;
```

```

if ($token eq '-') {
    return -parse_factor($tokens);
} elsif ($token eq '+') {
    return parse_factor($tokens);
} elsif ($token eq '(') {
    my $val = parse_expr($tokens);
    die "Ошибка: нет закрывающей скобки\n"
        unless @$tokens && shift(@$tokens) eq ')';
    return $val;
} elsif ($token =~ /\^d+(?:\.\d+)?$/) {
    return $token + 0;
} else {
    die "Ошибка: неверный токен '$token'\n";
}
}

sub evaluate {
    my ($line) = @_;
    my $tokens = tokenize($line);
    die "Ошибка: пустое выражение\n" unless @$tokens;
    my $result = parse_expr($tokens);
    die "Ошибка: синтаксический мусор в конце\n" if @$tokens;
    return $result;
}

print "Console Calculator (Perl)\n";
print "Enter expression or 'exit' to quit.\n\n";

while (1) {
    print "calc> ";
    my $line = <STDIN>;
    last unless defined $line;
    $line =~ s/\^s+|\s+$//g;
    next if $line eq "";
    last if $line =~ /\^(?:exit|quit)$/i;

    eval {
        my $res = evaluate($line);
        print "Result: $res\n";
    };
    if ($@) {
        my $err = $@;
        $err =~ s/ at .* line \d+\.\n$//;
        print "$err";
    }
}

```

Расчет метрик вручную с результатами

К операторам относятся:

- знаки операций, присваивания и разыменования (+, -, *, /, =, =~, ->, ? :)
- логические и строковые операторы-слова (eq, ne, and, or)
- разделители: точка с запятой и запятая (, ;)
- скобки: круглые () (только при выделении подвыражений), квадратные [] (индексация) и фигурные {} (блоки кода, аналог Begin...End)
- управляющие и составные конструкции (if...elsif...else, while, unless, return, last)
- имена встроенных процедур и пользовательских функций

К операндам относятся:

- переменные всех типов (\$val, @tokens, @_, \$@), числовые и строковые литералы, шаблоны регулярных выражений (/S/g) и дескрипторы потоков (STDIN).

Исключаются из анализа:

- директивы компилятора (use strict;, use warnings;), служебные слова объявлений (my, sub), однострочные комментарии и метки.

Таблица с результатами расчета метрик вручную

j	Оператор	f1j	i	Операнд	f2i
1	;	50	1	\$tokens	26
2	=	21	2	\$val	12
3	{}	19	3	\$line	8
4	eq	12	4	0	7
5	()	9	5	\$rhs	7
6	return	8	6	\$expr	6
7	=~	7	7	\$token	6
8	if...elsif...else	7	8	@_	5
9	die	7	9	\$op	5
10	print	5	10	1	4
11	parse_factor	5	11	@tokens	4
12	[]	4	12	'+'	3
13	&&	4	13	'.'	2
14	->	4	14	'/'	2
15	,	4	15	'*'	2
16	shift	4	16	\$pos	2
17	while	4	17	\$result	2
18	unless	4	18	\$@	2
19		3	19	\$err	2
20	parse_term	3	20	/S/g	1
21	parse_expr	3	21	/G(\d+(?:\.\d+)?)\s/gc	1

22	+	2	22	$\wedge G([\backslash + \backslash - \backslash * \backslash \wedge ()]) / g c$	1
23	-	2	23	"Ошибка: неверный символ ""	1
24	.	2	24	"" на позиции \$pos\n"	1
25	?	2	25	"Ошибка: деление на ноль\n"	1
26	:	2	26	"Ошибка: незавершённое выражение\n"	1
27	tokenize	2	27	'('	1
28	evaluate	2	28	"Ошибка: нет закрывающей скобки\n"	1
29	push	2	29	)'	1
30	last	2	30	$/\wedge d+(?:\backslash .\backslash d+)?\$/$	1
31	*	1	31	"Ошибка: неверный токен '\$token'\n"	1
32	\	1	32	"Ошибка: пустое выражение\n"	1
33	/	1	33	"Ошибка: синтаксический мусор в конце\n"	1
34	<>	1	34	"Console Calculator (Perl)\n"	1
35	==	1	35	"Enter expression or 'exit' to quit.\n\n"	1
36	defined	1	36	"calc> "	1
37	substr	1	37	STDIN	1
38	eval	1	38	$s/\wedge s+ \backslash s+\$/g$	1
39	next	1	39	"	1
40	pos	1	40	$/\wedge(?:exit quit)\$/i$	1
			41	\$res	1
			42	"Result: \$res\n"	1
			43	$s/ at . * line \backslash d+ \backslash .\backslash n\$/$	1
			44	"\$err"	1
η1=40		N1=215	η2=44		N2=132

η1=41

N1=212

η2=44

N2=132

Словарь программы **η** = 40 + 44 = 84

Длина программы **N** = 215 + 132 = 347

Объем программы **V** = 347 * log₂(84) = 347 * 6.4 = 2218

Скриншот с результатами работы программы

